This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

# Parametric Point Cloud Completion for Polygonal Surface Reconstruction

Zhaiyu Chen[1,3] Yuqing Wang[1] Liangliang Nan[2] Xiao Xiang Zhu[1,3]
[1]Technical University of Munich [2]Delft University of Technology [3]Munich Center for Machine Learning

## Abstract

*Existing polygonal surface reconstruction methods heavily depend on input completeness and struggle with incomplete point clouds. We argue that while current point cloud completion techniques may recover missing points, they are not optimized for polygonal surface reconstruction, where the parametric representation of underlying surfaces remains overlooked. To address this gap, we introduce parametric completion, a novel paradigm for point cloud completion, which recovers parametric primitives instead of individual points to convey high-level geometric structures. Our presented approach, PaCo, enables high-quality polygonal surface reconstruction by leveraging plane proxies that encapsulate both plane parameters and inlier points, proving particularly effective in challenging scenarios with highly incomplete data. Comprehensive evaluations of our approach on the ABC dataset establish its effectiveness with superior performance and set a new standard for polygonal surface reconstruction from incomplete data. Project page:* https://parametric-completion.github.io.

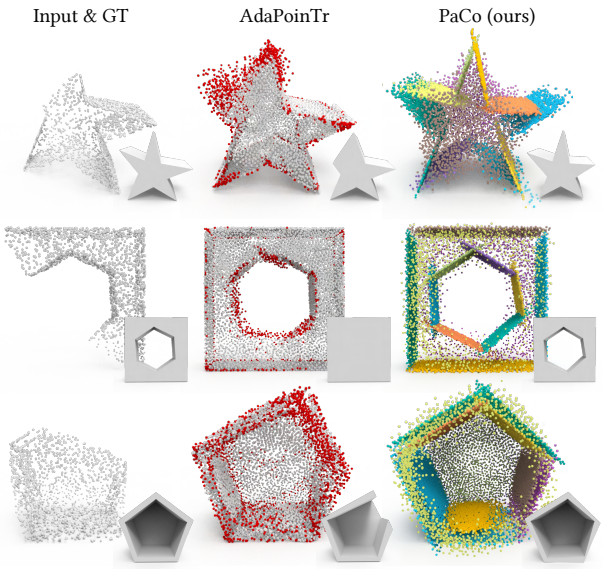



Figure 1. Unlike conventional point cloud completion that aims to recover individual points (*middle*, non-planar points shown in red), parametric completion recovers primitive parameters along with corresponding inlier points. PaCo recovers planar structures (*right*, each color denoting a planar primitive), *directly* enabling high-quality polygonal surface reconstruction.

## 1. Introduction

Unlike conventional methods that produce dense triangle meshes [12, 18, 19], polygonal surface reconstruction prioritizes a compact surface representation [1, 26, 32]. The compact polygonal representation is valuable for applications where efficiency, interpretability, and editability are essential [2]. However, recovering accurate polygonal surfaces from incomplete point clouds remains a challenge due to factors such as occlusions and sensor limitations. This demands methods that can infer structured, high-level surface representations (*e.g.*, planes) to effectively convey underlying geometries.

One might consider point cloud completion as a potential solution. However, current practices in point cloud completion aim to recover individual missing points [46–48]. While existing methods show promise for free-form object completion, they typically fail to produce structured reconstructions with piecewise planar geometry. The emphasis on point completeness in existing methods overlooks the need for a parametric planar representation, which is essential for accurately reconstructing polygonal surfaces. We argue that a persistent gap exists between the goals of point cloud completion and polygonal surface reconstruction. To our knowledge, none of the existing completion techniques are optimized for polygonal surfaces.

To bridge this gap, we introduce ***parametric point cloud completion, a novel point cloud completion paradigm***. Unlike conventional methods that emphasize individual point recovery to improve completeness, parametric completion aims to infer parametric primitives from incomplete data. This shift prioritizes the recovery of high-level geometric structures over point completeness. We present PaCo, a neural network that embodies this paradigm and leverages sequence-to-sequence generation to learn plane prox-

ies. The learned proxies convey the underlying structures of complete surfaces from incomplete inputs, encoding both plane parameters and corresponding inlier points. We then formulate a bipartite matching framework with multiple objectives to optimize primitive distribution. By leveraging these parametric primitives, PaCo effectively enables the reconstruction of high-quality polygonal surfaces from incomplete data, even in challenging scenarios where conventional completion methods typically fail. Fig. 1 highlights the distinct advantages of PaCo over existing methods.

Benefiting from our parametric representation and associated learning strategies, our approach bridges the gap between shape completion and polygonal surface reconstruction. We summarize our contributions as follows:

- We introduce parametric point cloud completion, a new completion paradigm aimed at inferring parametric primitives that convey the underlying structures of object surfaces from incomplete input data.
- We present PaCo, a novel deep neural network that embodies the parametric completion paradigm for polygonal surfaces.
- We develop a bipartite matching framework with multiple learning objectives that optimally distributes parametric primitives for accurate and structured completion.
- We establish the effectiveness of parametric completion for reconstructing polygonal surfaces from incomplete point clouds, setting a new standard for this task.

## 2. Related Work

**3D Shape Completion.** Early 3D shape completion methods often rely on volumetric grid data structures [11, 38, 40], applying convolutions to process the data. However, converting point clouds into voxels introduces discretization errors and incurs high computational costs at fine resolutions. To enable direct operation on raw point sets, PointNet [27] utilizes symmetric functions to achieve permutation invariance. This breakthrough led to the development of various point-based neural networks [28, 35, 37] that efficiently consume point cloud data. Point-based methods have since become the mainstream for point cloud completion, with models like PCN [48], FoldingNet [42], PoinTr [46], AdaPoinTr [47], ODGNet [4], and many more [24, 33, 34, 39, 41, 49, 50] demonstrating effectiveness in filling missing points. However, these methods often overlook the underlying geometric structures of surfaces while focusing on individual point recovery. In contrast, our approach emphasizes recovering a parametric planar representation that captures higher-level geometric structures essential for polygonal surfaces.

**Polygonal Surface Reconstruction.** Polygonal surfaces can be reconstructed using primitive assembly methods, which optimize the arrangement of planar primitives or their equivalents. Key examples include PolyFit [26], KSR [1], COMPOD [32], and domain-specific ones [7, 9, 16]. These methods rely on high-quality primitives, often extracted using techniques such as RANSAC [31] and GoCoPP [45]. An alternative approach involves geometric simplification, which reduces mesh complexity while retaining key geometric features [8, 10, 13, 14, 22, 30]. Geometric simplification typically begins with a dense mesh generated by methods like Poisson [19] or neural solvers [3, 12, 15, 17]. However, geometric simplification requires high-quality input meshes and typically underperforms compared to primitive assembly to achieve accurate approximations. Notably, neural methods have also shown potential in reconstructing parametric surface meshes [6, 23, 25, 29, 43] but struggle with incomplete data, which are common due to occlusions and sensor limitations [18]. These limitations motivate our approach: deriving a parametric representation directly from incomplete point clouds to facilitate robust polygonal surface reconstruction via primitive assembly.

## 3. Method

Given an incomplete point cloud $X$, our objective is to infer *all* parametric planar primitives $\hat{P}$ that constitute the complete surface of the object, with each primitive representing a piecewise planar region. We start by hierarchically encoding the input points into plane proxies $V$ that represent the planar primitives in the input. These proxies are then fed to a proxy generator to produce a set of proxy proposals for the entire surface. Next, a parametric recovery module extracts the plane parameters and distributes their inlier points from these proxies. Unlike existing completion methods that predict a fixed number of points, PaCo uses variable-size primitives that adapt to surfaces of different complexities. This is achieved through bipartite matching during training, while during inference, primitives are selected based on confidence scores from a primitive selector. Fig. 2 presents the overall architecture of PaCo.

### 3.1. Hierarchical Encoding

Given $X = \{x_i\}_{i=1}^{N_x}$, we first group the points into a set of planar segments $S = \{s_i\}_{i=1}^{N_s}$ using GoCoPP [45], represented by a surjective mapping $f : X \to S$. Our encoder progressively builds features across three topological levels: point, point patch, and plane, guided by this mapping.

To start, the points are encoded as point proxies $X'$ following the PoinTr encoding scheme [46]. Instead of encoding individual points, these point proxies represent features of local point patches, with each patch's location defined by its center point in the incomplete point cloud. Since the geometry of $X$ covers $X'$, we establish a lossless mapping $f' : X \to X' \to S$ using a lookup table derived from the mapping $f$, allowing hierarchical feature aggregation to the plane level. Finally, we inject each plane's normal em-

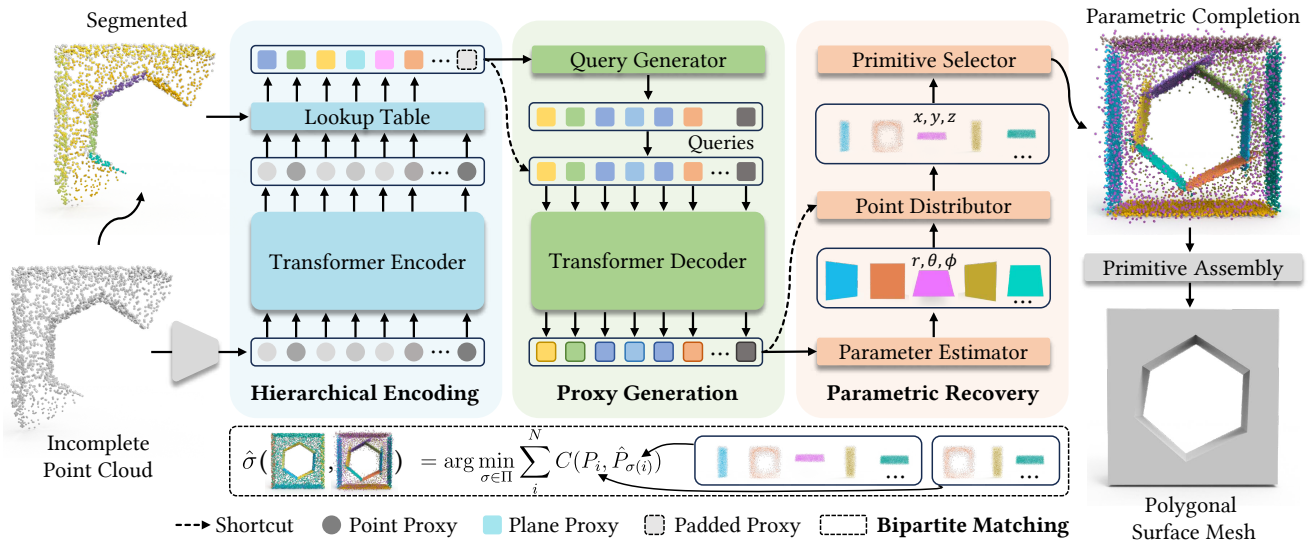



Figure 2. **Architecture of PaCo.** Starting from an incomplete point cloud, we hierarchically encode the points into plane proxies with a lookup table from segmentation. These proxies inform the proxy generator in producing a set of proxy proposals. The generated proxies are then passed to the parameter estimator and the point distributor for primitive estimation. Finally, the primitive selector identifies a subset of primitives to form the parametric completion, with selection optimized via bipartite matching. The resulting primitives enable *direct* assembly for polygonal surface reconstruction. Points are colored according to their planar primitives.

bedding into the plane-level semantic feature, incorporating structural information into the plane proxies $V = \{v_i\}_{i=1}^{K}$. Thus, the $i$-th plane proxy $v_i$ is computed as

$$v_i = \mathit{sum}(X'_i) + \Phi(n_i), \tag{1}$$

where *sum* denotes aggregation with sum pooling, $X'_i \subseteq X'$ represents all point proxies on the plane, $\Phi$ is an MLP, and $n_i$ represents the normal of the plane.

### 3.2. Proxy Generation

The proxy generation module takes the input plane proxies and generates a set of plane proxy proposals, which complete the surface of the object by filling in planes that are partially missing or completely absent in the input.

Since $V$ varies in length, we first pad it to a fixed size and apply an MLP to produce a set of queries $Q_I$ representing primitives in the input. $V$ is then passed to a query generator, producing another set of queries $Q_G$ representing the missing primitives. We adopt the query ranking strategy [47] to select the top $M$ queries from both sources combined. A Transformer block [36] then attends to the input plane proxies $V$ and the selected queries $Q = \{q_i\}_{i=1}^{M}$ to generate a set of plane proxies as proposals.

### 3.3. Parametric Recovery

The generated proxies are fed into the parametric recovery module to retrieve plane parameters and inlier points, using three prediction heads: parameter estimator, point distributor, and primitive selector.

**Parameter Estimator.** The parameter estimator, implemented as an MLP, takes the plane proxies as input to predict plane parameters. Each plane is represented in polar coordinates $(r_i, \theta_i, \varphi_i)$ instead of the Cartesian representation, to avoid degeneracies for planes parallel to the axes.

**Point Distributor.** Parameters alone do not define a primitive. Here, we predict the distribution of points on each plane. Leveraging the $M$ plane parameters generated by the parameter estimator, we use a point distributor head to reconstruct the point distribution on the predicted planes. Specifically, we estimate the polar angles $(\theta_{ij}, \varphi_{ij})$ of every point, while the radius is derived from these angles and the plane parameters:

$$r_{ij} = \frac{r_i}{\cos(\Delta\varphi)\sin(\theta_{ij})\sin(\theta_i) + \cos(\theta_{ij})\cos(\theta_i)}, \tag{2}$$

where $\Delta\varphi = \varphi_{ij} - \varphi_i$, and $(r_{ij}, \theta_{ij}, \varphi_{ij})$ represents the polar coordinates of the $j$-th point on the $i$-th plane.

**Primitive Selector.** As a fixed number of primitives are predicted, not all will comprise the surface. For each predicted plane proxy, the primitive selector assigns a confidence score $\kappa_i \in [0, 1]$, indicating the probability that the primitive is part of the object surface. This score helps distinguish surface-contributing primitives from non-contributing ones. During training, only the selected planar primitives are supervised as detailed in Sec. 3.4.

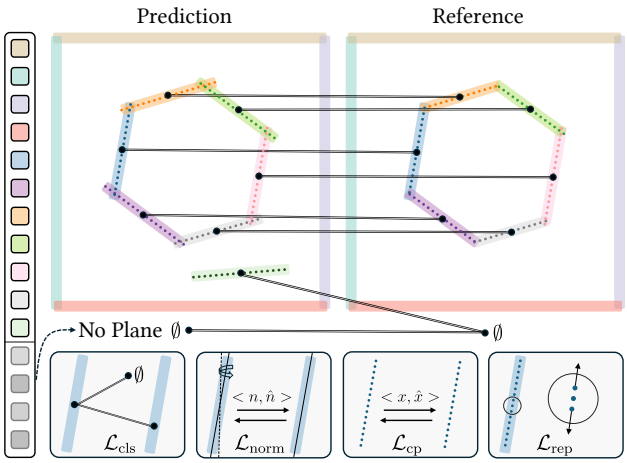



Figure 3. **Optimization objectives.** Our bipartite matching incorporates four objectives: primitive classification loss ($\mathcal{L}_{\text{cls}}$) for primitive selection, plane normal loss ($\mathcal{L}_{\text{norm}}$) for plane parameter estimation, plane chamfer loss ($\mathcal{L}_{\text{cp}}$) for inlier alignment, and repulsion loss ($\mathcal{L}_{\text{rep}}$) for uniform point distribution. Some pairs are omitted for brevity.

### 3.4. Set Matching and Optimization

The ground truth consists of a set of unordered planar primitives of varying sizes. Given the predicted primitives, we compare them against the ground truth by constructing a bipartite matching graph between the two sets and applying loss functions to the matched pairs. The network is then optimized using these objectives. Fig. 3 illustrates the main optimization objectives.

**Bipartite Matching.** We establish correspondences between ground truth planar primitives $P = \{p_i\}_{i=1}^{M}$ and predicted primitives $\hat{P} = \{\hat{p}_i\}_{i=1}^{M}$ using bipartite matching [5]. The cost matrix for matching is aligned with the loss function to promote stable training. For each matched pair of predicted and ground truth primitives, the matching cost between ground truth primitive $p_i$ and the prediction at index $\sigma(i)$ is computed as

$$C(p_i, \hat{p}_{\sigma(i)}) = \underbrace{\mathcal{L}_{\text{cls}}}_{\text{semantic}} + \underbrace{\beta_1 \mathcal{L}_{\text{norm}} + \beta_2 \mathcal{L}_{\text{cp}} + \beta_3 \mathcal{L}_{\text{rep}}}_{\text{geometric}}. \tag{3}$$

The geometric cost incorporates plane normal loss $\mathcal{L}_{\text{norm}}$, plane completion loss $\mathcal{L}_{\text{cp}}$, and repulsion loss $\mathcal{L}_{\text{rep}}$, while the semantic cost is defined by the primitive classification loss $\mathcal{L}_{\text{cls}}$. The coefficients $\beta_1$, $\beta_2$, and $\beta_3$ weight each respective loss term.

We represent the ground truth planar primitives as a set, padding it with $\emptyset$ to match the cardinality of the predicted set, where $\emptyset$ represents the absence of a planar primitive. To establish bipartite matching between these two sets, we search for a permutation $\sigma \in \Pi$ that minimizes the total matching cost:

$$\hat{\sigma} = \arg\min_{\sigma \in \Pi} \sum_{i}^{M} C(p_i, \hat{p}_{\sigma(i)}). \tag{4}$$

This optimal assignment is efficiently computed using the Hungarian algorithm [21].

**Loss Function.** Our loss function quantifies discrepancies between prediction and ground truth across three aspects: primitive selection, plane parameter estimation, and point completion. It consists of the following terms:

• *Primitive classification loss* encourages the selection of primitives contributing to the surface while discouraging those not contributing. We define this loss for all predicted primitives as

$$\mathcal{L}_{\text{cls}} = -\mathbb{1}_{\{\hat{c}_i=1\}} \cdot \log \kappa_{\hat{\sigma}(i)} - \mathbb{1}_{\{\hat{c}_i=0\}} \cdot \log(1 - \kappa_{\hat{\sigma}(i)}), \tag{5}$$

where $\kappa_{\hat{\sigma}(i)}$ represents the confidence score predicted by the primitive selector, and $\mathbb{1}$ is an indicator function that equals 1 if the predicted primitive corresponds to a target primitive and 0 otherwise.

• *Plane normal loss* penalizes the discrepancy between predicted and ground truth plane normals, defined as

$$\begin{aligned}\mathcal{L}_{\text{norm}} = \mathbb{1}_{\{\hat{c}_i=1\}} \cdot \lambda \left(1 - \cos\left(\angle\left(n_i, \hat{n}_{\hat{\sigma}(i)}\right)\right)\right) + \\ \mathbb{1}_{\{\hat{c}_i=1\}} \cdot \|n_i - \hat{n}_{\hat{\sigma}(i)}\|^2,\end{aligned} \tag{6}$$

where $\ell_2$ distance and cosine similarity are used to measure their similarity, and $\lambda$ is a scaling factor.

• *Plane chamfer loss* aligns inlier points on each matched primitive pair by assessing the chamfer distance (*CD*):

$$\mathcal{L}_{\text{cp}} = \mathbb{1}_{\{\hat{c}_i=1\}} \cdot CD\left(p_i, \hat{p}_{\sigma(i)}\right). \tag{7}$$

• *Overall chamfer loss* further optimizes point distribution. Unlike $\mathcal{L}_{\text{cp}}$, which applies to individual planar primitives, this loss term applies to the entire object surface:

$$\mathcal{L}_{\text{co}} = CD\left(\{p_i \mid c_i = 1\}, \{\hat{p}_i \mid \hat{c}_i = 1\}\right), \tag{8}$$

where we aggregate all points from selected primitives and compute their chamfer distance to the ground truth.

• *Repulsion loss* promotes a more uniform point distribution, countering the tendency of points to cluster around the projected center of the plane when trained solely with the plane chamfer loss. The repulsion loss [44] is defined as

$$\mathcal{L}_{\text{rep}} = \sum_{i=1}^{T} \sum_{i' \in K(i)} -\|\hat{x}_{i'} - \hat{x}_i\| \cdot \exp(-\omega \|\hat{x}_{i'} - \hat{x}_i\|^2), \tag{9}$$

where $T$ is the number of predicted points on the primitive, $K(i)$ is the set of indices of the $k$-nearest neighbors of point $\hat{x}_i$, $\|\hat{x}'_i - \hat{x}_i\|$ represents the distance between generated points $\hat{x}_i$ and $\hat{x}_{i'}$, and $\omega$ is a constant factor.

The final loss is the sum of the above loss terms:

$$\mathcal{L}_{total} = \sum_{i=1}^{\mathrm{M}} \left( \mathcal{L}_{\mathrm{cls}}^{(i)} + \beta_1 \mathcal{L}_{\mathrm{norm}}^{(i)} + \beta_2 \mathcal{L}_{\mathrm{cp}}^{(i)} + \beta_3 \mathcal{L}_{\mathrm{rep}}^{(i)} \right) + \beta_4 \mathcal{L}_{\mathrm{co}}, \tag{10}$$

where $\beta_1$, $\beta_2$, and $\beta_3$ are the same as in Eq. (3), and the additional $\beta_4$ weights the overall chamfer loss.

# 4. Experiments

## 4.1. Experimental Setup

**Dataset.** To evaluate the performance of PaCo, we conduct experiments on the ABC dataset [20], a large-scale collection of CAD models for geometric deep learning. We select 15,339 CAD models with *plane-only* structures (see supplementary material). Following AdaPoinTr [47], we define three occlusion levels (*simple*, *moderate*, and *hard*) with point missing ratios of 25%, 50%, and 75%, respectively. Eight views are generated per model and occlusion level, yielding 368,136 point clouds. We reserve 2,375 CAD models and their associated point clouds for evaluation, ensuring strict separation from the training data. All data are normalized to a unit diagonal length.

**Evaluation Metrics.** To assess reconstruction fidelity, we use symmetric chamfer distance *(CD)*, Hausdorff distance *(HD)*, and normal consistency *(NC)*, computed from 10,000 sampled surface points. Additionally, the failure rate *(FR)* is reported as the proportion of unsolvable samples, with these samples evaluated against the unit cube. Unless otherwise specified, models are trained for 200 epochs for ablation studies (Sec. 4.6) and 600 epochs for all other experiments.

## 4.2. Evaluation against Completion

Since the completion is dedicated to surface reconstruction, we benchmark our completion scheme against state-of-the-art point cloud completion methods by feeding the completed points to polygonal surface reconstruction solvers and evaluating the reconstructed surfaces. In this evaluation, we include six point cloud completion methods, PCN [48], FoldingNet [42], GRNet [40], PoinTr [46], AdaPoinTr [47], and ODGNet [4], along with three polygonal surface reconstruction solvers, PolyFit [26], KSR [1], and COMPOD [32]. Unlike the competing methods that lack explicit structures in their predictions, PaCo generates parametric planar primitives that can be *directly* used by the solvers, while GoCoPP [45] is used to extract planar primitives for the other completion methods.

Tab. 1 presents the quantitative results. Our method outperforms others across *all* fidelity metrics by substantial margins and also achieves the *lowest* failure rate, regardless of the reconstruction solver used. Fig. 4 provides qualitative results with the PolyFit solver: our method produces surface geometries closest to the ground truth, whereas other completion methods frequently miss structures or even completely fail. PaCo's performance is also the *most consistent* across different occlusion levels.

## 4.3. Evaluation against Reconstruction

In addition to the completion-and-reconstruction comparison, we also directly compare our approach with both traditional and recent neural reconstruction methods without the point cloud completion step. This evaluation helps to understand the role of completion in achieving faithful surface reconstruction. The traditional methods involved in this analysis include PolyFit [26], KSR [1], and COMPOD [32], while the neural methods include BSP-Net [6] and SECAD-Net [23]. Since both neural methods require voxel input, we prepare voxels with the same level of incompleteness for consistency. For our approach, we choose the PolyFit solver as it shows strong performance in Tab. 1.

Tab. 2 presents the quantitative comparison. All traditional reconstruction methods fail on a significant portion of inputs, hindered by the missing structures in the input points. While the two neural reconstruction methods manage to produce some geometry, they still fail to capture the main structures of the object surfaces. Fig. 5 shows qualitative examples, where BSP-Net tends to produce over-simplified meshes, and both competing methods struggle to generalize. In contrast, our method demonstrates the *strongest* capability in structure recovery. This comparison also reveals that direct reconstruction of polygonal surfaces from incomplete data is less effective than the completion-and-reconstruction strategy, as evidenced by the performance gap between Tab. 1 and Tab. 2.

## 4.4. Evaluation against Simplification

We further compare our approach with two geometric simplification methods: the classic QEM [14] and the recent RoLoPM [8]. We use PolyFit as the reconstruction solver for our approach, and ODGNet [4] to complete the points for Poisson reconstruction [19], given their strong performance in Tab. 1. Simplification is then applied to the Poisson results. In addition to mesh fidelity, we report face count ($\#f$) and vertex count ($\#v$) as indicators of geometric complexity. To ensure a fair comparison, we set the target face counts for the simplification methods to closely match our triangulated results.

Tab. 3 presents the quantitative results on 50 randomly selected samples, where our approach achieves the *highest* performance while using the *fewest* faces. Fig. 6 provides a qualitative comparison with the two simplification methods, showing that our result has significantly greater regularity.

Table 1. **Quantitative comparison with conventional point cloud completion methods.** Evaluation is conducted on the reconstructed meshes. **Bold** numbers indicate the best results, underlined the second best. Our method consistently outperforms others across nearly all metrics regardless of the reconstruction solver used. Note: CD, HD, and FR values are scaled by a factor of 100.

| Method | PolyFit [26] | | | | COMPOD [32] | | | | KSR [1] | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | CD ↓ | HD ↓ | NC ↑ | FR ↓ | CD ↓ | HD ↓ | NC ↑ | FR ↓ | CD ↓ | HD ↓ | NC ↑ | FR ↓ |
| PCN [48] | 14.10 | 20.73 | 0.620 | 71.27 | 16.27 | 22.20 | 0.562 | 97.87 | 21.61 | 44.23 | 0.691 | 11.90 |
| FoldingNet [42] | 12.07 | 21.24 | 0.814 | 3.54 | 7.40 | 18.60 | 0.805 | **0.04** | 18.11 | 42.59 | 0.693 | 1.86 |
| GRNet [40] | 11.98 | 19.84 | 0.769 | 29.61 | 14.19 | 25.72 | 0.738 | 13.15 | 9.18 | 22.01 | 0.822 | 10.82 |
| PoinTr [46] | 10.57 | 16.43 | 0.822 | 25.92 | 7.82 | 16.44 | 0.774 | 31.34 | 8.14 | 16.33 | 0.780 | 30.90 |
| AdaPoinTr [47] | 3.16 | 7.36 | 0.920 | 5.89 | 3.25 | 8.84 | 0.921 | 1.32 | 3.24 | 8.86 | 0.927 | 0.27 |
| ODGNet [4] | 2.73 | 6.41 | 0.933 | 4.28 | 3.22 | 8.01 | 0.927 | 1.05 | 2.90 | 8.56 | 0.934 | 0.36 |
| PaCo (ours) | **1.87** | **4.09** | **0.943** | **0.48** | **1.94** | **4.42** | **0.940** | 0.25 | **1.91** | **4.14** | **0.940** | **0.25** |

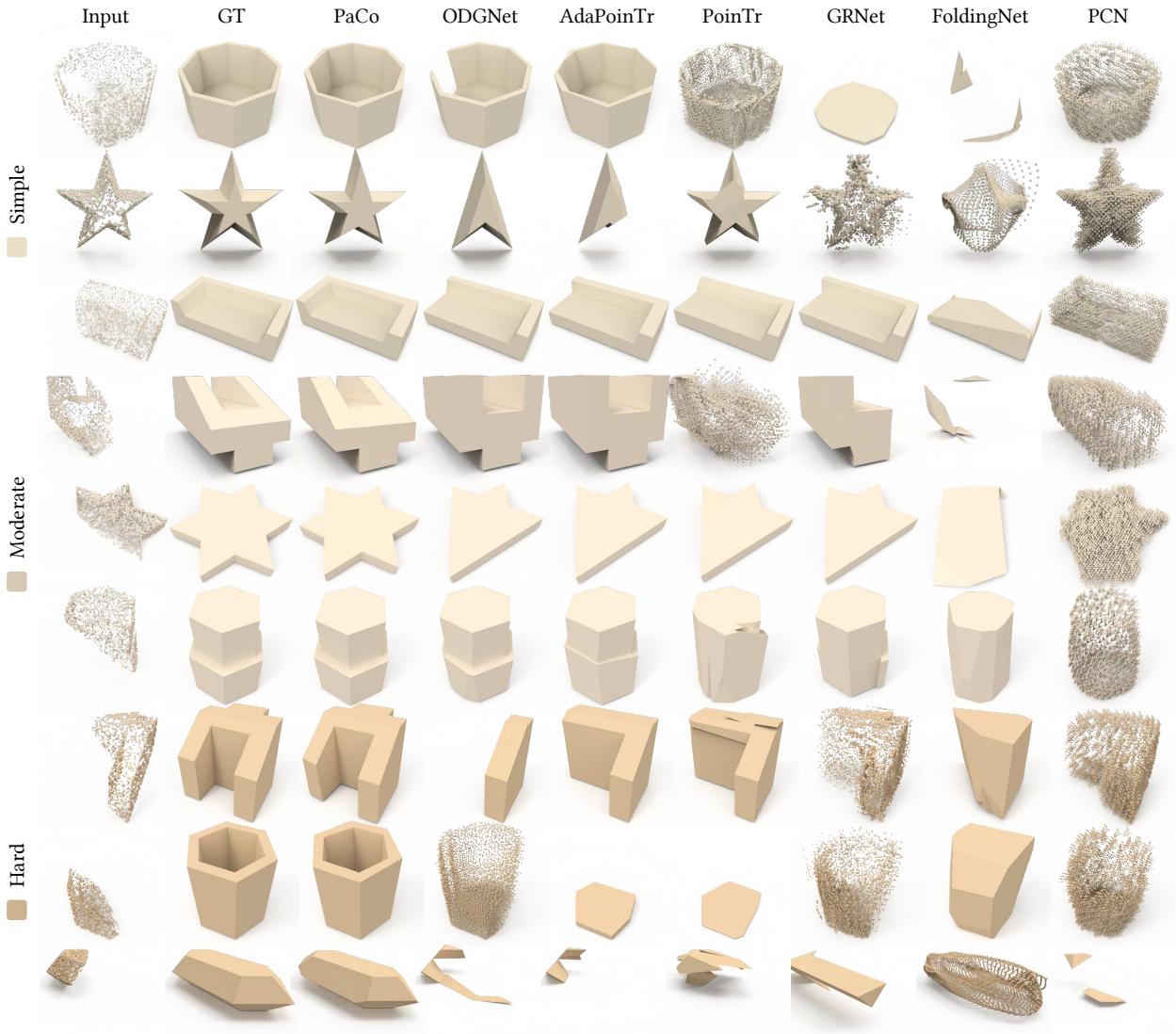



Figure 4. **Qualitative comparison with conventional point cloud completion methods.** Simple, moderate, and hard denote missing ratios of 25%, 50%, and 75%, respectively. For cases where reconstruction fails, the completed points are shown instead. Our method excels in recovering geometric structures with planar primitives directly usable by the reconstruction solver.

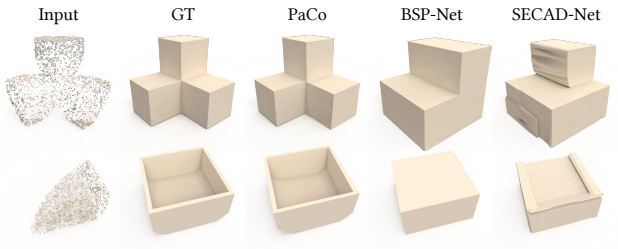



Figure 5. **Qualitative comparison with neural reconstruction methods directly from incomplete inputs**. Our method demonstrates stronger capability in terms of structure recovery.

Table 2. **Quantitative comparison with reconstruction methods.** Neural approaches are in gray. Traditional methods fail on a significant portion of inputs, while the competing neural methods struggle to preserve structural integrity, both underperform our completion-and-reconstruction strategy.

| Method | CD ↓ | HD ↓ | NC↑ | FR ↓ |
|---|---|---|---|---|
| PolyFit [26] | 15.11 | 24.26 | 0.618 | 63.28 |
| KSR [1] | 9.18 | 22.01 | 0.822 | 10.82 |
| COMPOD [32] | 6.87 | 18.88 | 0.826 | 9.64 |
| BSP-Net [6] | 7.69 | 20.14 | 0.652 | **0.00** |
| SECAD-Net [23] | 4.35 | 11.33 | 0.850 | **0.00** |
| PaCo (ours) | **1.87** | **4.09** | **0.943** | 0.48 |

Table 3. **Quantitative comparison with simplification methods**. $\#f$ and $\#v$ denote the number of faces and vertices, respectively. All metrics are measured on 50 randomly selected samples. "39 tri." denotes the average triangle number after triangulation.

| Method | $\#f$ | $\#v$ | CD ↓ | HD ↓ | NC ↑ |
|---|---|---|---|---|---|
| QEM [14] | 39 | 22 | 2.67 | 11.32 | 0.900 |
| RoLoPM [8] | 44 | 23 | 2.43 | 9.05 | 0.918 |
| PaCo (ours) | **13** (39 tri.) | 22 | **1.72** | **4.13** | **0.934** |

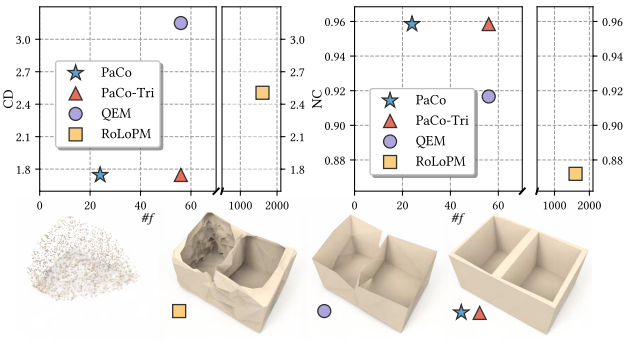



Figure 6. **Qualitative comparison with simplification methods.** Simplifications are applied to the Poisson reconstruction from the completed points. PaCo-Tri denotes ours triangulated. Our result demonstrates greater regularity and has the fewest faces.

Table 4. **Quantitative evaluation on robustness against noise.** Ratios denote Gaussian noise defined relative to the object's diagonal length. Models are trained on respective noise levels.

| Noise | CD ↓ | HD ↓ | NC ↑ | FR ↓ |
|---|---|---|---|---|
| 4.5% | 3.75 | 7.97 | 0.872 | 0.65 |
| 3.0% | 2.32 | **5.06** | 0.925 | 0.48 |
| 1.5% | 2.29 | 5.07 | 0.925 | **0.44** |
| 0.0% | **2.11** | 5.26 | **0.941** | 0.48 |

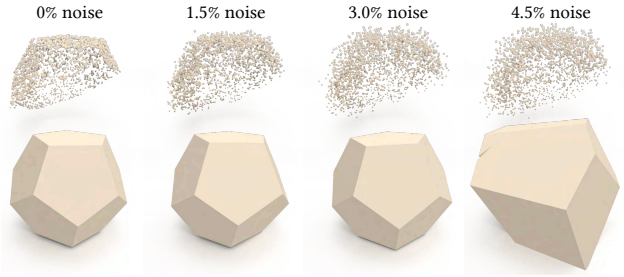



Figure 7. **Qualitative example on robustness against noise.** Reliable reconstruction is obtained, up to a prohibitive noise level of 4.5% Gaussian noise where structural changes become evident.

### 4.5. Robustness Analysis

**Noise.** Tab. 4 presents quantitative results of our method under different noise levels, with Fig. 7 showing a series of qualitative examples. Note that models are trained at respective noise levels for 200 epochs, with initial planar structures intact. Gaussian noise at 1.5% of the object's diagonal length results in a slight increase in errors, while raising the noise to 3.0% does not significantly degrade performance. The failure rate remains stable as noise increases, up to a prohibitive level of 4.5% where structural changes become evident. These results reinforce our focus on parametric recovery over completing individual points.

**Incompleteness.** Tab. 5 quantitatively compares our method with others across different levels of incompleteness. At the *simple* incompleteness level, the performance gain of our method is relatively modest. However, on more challenging data featuring larger missing areas (*moderate* and *hard*), all competing methods show a steep decline in performance, while ours remains largely stable. This robustness results in the pronounced performance advantage on the *hard* samples and superior overall performance. Fig. 4 visually demonstrates this trend: with greater incompleteness, our method continues to produce plausible reconstructions, whereas other methods struggle to preserve structural integrity as they primarily focus on recovering individual points. In the supplementary material, we show that PaCo also effectively processes real-scanned airborne LiDAR data for building reconstruction.

Table 5. **Performance comparison in terms of data incompleteness.** Our method performs competitively on simple data (S, -25%), and excels on moderate (M, -50%) and hard (H, -75%) data.

| Method | CD ↓ | | | NC ↑ | | |
|---|---|---|---|---|---|---|
| | S | M | H | S | M | H |
| PCN [48] | 13.06 | 13.59 | 15.65 | 0.654 | 0.631 | 0.577 |
| FoldingNet [42] | 11.22 | 11.67 | 13.33 | 0.840 | 0.825 | 0.776 |
| GRNet [40] | 10.23 | 11.76 | 13.97 | 0.849 | 0.780 | 0.679 |
| PoinTr [46] | 10.05 | 10.39 | 11.28 | 0.846 | 0.830 | 0.789 |
| AdaPoinTr [47] | 1.55 | 2.12 | 5.81 | 0.965 | 0.949 | 0.846 |
| ODGNet [4] | **1.48** | 2.01 | 4.72 | **0.969** | 0.950 | 0.879 |
| PaCo (ours) | 1.52 | **1.66** | **2.43** | 0.959 | **0.952** | **0.919** |

Table 6. **Ablation of network components.** Our method is optimally configured with polar coordinate representation, plane proxies, and sum pooling aggregation. "-" denotes not applicable.

| Component | | | CD ↓ | HD ↓ | NC ↑ |
|---|---|---|---|---|---|
| Polar | Plane Proxy | Aggregation | | | |
| | ✓ | sum | 3.34 | 7.87 | 0.879 |
| ✓ | | - | 2.45 | 6.49 | 0.916 |
| ✓ | ✓ | mean | 2.31 | 5.90 | 0.924 |
| ✓ | ✓ | sum | **2.11** | **5.26** | **0.941** |

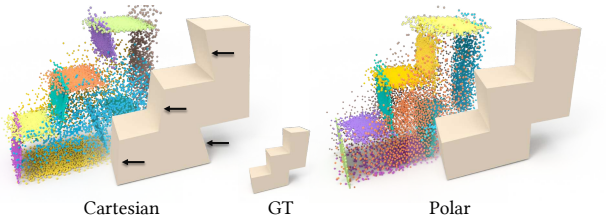



Figure 8. **Impact of coordinate representation.** Our polar representation facilitates the formulation of axis-aligned planar primitives, producing structures of greater regularity than that of the Cartesian counterpart.

### 4.6. Ablation Studies

**Network Architecture.** Tab. 6 presents the contributions of key components in our network architecture. Replacing the polar representation with the commonly used Cartesian representation results in a substantial performance drop. Fig. 8 visually compares these representations, showing that the polar form enables the formation of primitives perpendicular to the ground, producing structures with greater regularity. In contrast, the Cartesian representation struggles to achieve this structural consistency. Similarly, removing plane proxies in the encoding hierarchy hinders performance, as point proxies alone cannot provide structured feature aggregation as needed. Lastly, substituting sum with mean pooling in the aggregation step leads to the loss of scale information across proxies, resulting in a marked performance reduction.

Table 7. **Ablation of loss terms.** Our method performs the best when the repulsion loss ($\mathcal{L}_{\text{rep}}$), overall chamfer loss ($\mathcal{L}_{\text{co}}$), and plane normal loss ($\mathcal{L}_{\text{norm}}$) are all present.

| Term | | | CD ↓ | HD ↓ | NC ↑ |
|---|---|---|---|---|---|
| $\mathcal{L}_{\text{rep}}$ | $\mathcal{L}_{\text{co}}$ | $\mathcal{L}_{\text{norm}}$ | | | |
| | ✓ | ✓ | 2.48 | 6.37 | 0.916 |
| ✓ | | ✓ | 2.45 | 6.15 | 0.918 |
| ✓ | ✓ | | 2.22 | 5.42 | 0.928 |
| ✓ | ✓ | ✓ | **2.11** | **5.26** | **0.941** |

**Loss Terms.** Tab. 7 further presents the contributions of the optional loss terms $\mathcal{L}_{\text{rep}}$, $\mathcal{L}_{\text{co}}$, and $\mathcal{L}_{\text{norm}}$ in Eq. (10). Removing any of these terms leads to a significant and consistent performance drop across all metrics. This reveals the importance of maintaining a repulsive and reference-aligned point distribution and emphasizes the critical role of accurate plane parameter estimation.

## 5. Conclusion

We have introduced parametric completion, a new paradigm for point cloud completion, and presented PaCo, a novel framework that leverages this paradigm to address the challenges of polygonal surface reconstruction. Unlike conventional methods that prioritize individual point recovery, PaCo recovers structured, parametric planar primitives, enabling accurate and robust reconstruction of polygonal surfaces from incomplete point cloud data. This paradigm shift from point-based to parametric representation bridges the gap between shape completion and polygonal surface reconstruction and establishes a new standard for recovering structured geometries from incomplete data.

**Limitations.** PaCo is designed specifically for polygonal surface reconstruction and is not optimized for conventional point cloud completion tasks that prioritize the distribution of individual points. Its hierarchical encoding is based on the presence of planar structures in the input, which can limit the ability of PaCo to form meaningful proxies if such structures are absent. However, we did not encounter such failures in our experiments, even in challenging scenarios. Besides, the current implementation of PaCo is restricted to point clouds comprising only planar primitives, and it may fail for objects with non-planar geometries.

**Future Work.** A natural extension of PaCo would be to incorporate additional parametric shape primitives, such as spheres and cylinders. We anticipate that this extension will further strengthen the connection between point cloud completion and surface reconstruction, advancing toward a unified framework for recovering complex object geometries from incomplete data.

## Acknowledgment

This work was supported by TUM Georg Nemetschek Institute under the AI4TWINNING project. We thank anonymous reviewers for their constructive comments.

## References

[1] Jean-Philippe Bauchet and Florent Lafarge. Kinetic shape reconstruction. *ACM TOG*, 39(5):1–14, 2020. 1, 2, 5, 6, 7

[2] Mario Botsch, Leif Kobbelt, Mark Pauly, Pierre Alliez, and Bruno Lévy. *Polygon mesh processing*. AK Peters/CRC Press, Natick, MA, USA, 2010. 1

[3] Alexandre Boulch and Renaud Marlet. POCO: Point convolution for surface reconstruction. In *CVPR*, pages 6302–6314, 2022. 2

[4] Pingping Cai, Deja Scott, Xiaoguang Li, and Song Wang. Orthogonal dictionary guided shape completion network for point cloud. In *AAAI*, pages 864–872, 2024. 2, 5, 6, 8

[5] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *ECCV*, pages 213–229, 2020. 4

[6] Zhiqin Chen, Andrea Tagliasacchi, and Hao Zhang. BSP-Net: Generating compact meshes via binary space partitioning. In *CVPR*, pages 45–54, 2020. 2, 5, 7, 1

[7] Zhaiyu Chen, Hugo Ledoux, Seyran Khademi, and Liangliang Nan. Reconstructing compact building models from point clouds using deep implicit fields. *ISPRS Journal of Photogrammetry and Remote Sensing*, 194:58–73, 2022. 2

[8] Zhen Chen, Zherong Pan, Kui Wu, Etienne Vouga, and Xifeng Gao. Robust low-poly meshing for general 3D models. *ACM TOG*, 42(4):1–20, 2023. 2, 5, 7

[9] Zhaiyu Chen, Yilei Shi, Liangliang Nan, Zhitong Xiong, and Xiao Xiang Zhu. PolyGNN: Polyhedron-based graph neural network for 3D building reconstruction from point clouds. *ISPRS Journal of Photogrammetry and Remote Sensing*, 218:693–706, 2024. 2

[10] David Cohen-Steiner, Pierre Alliez, and Mathieu Desbrun. Variational shape approximation. In *ACM SIGGRAPH 2004 Papers*, page 905–914, New York, NY, USA, 2004. Association for Computing Machinery. 2

[11] Angela Dai, Charles R. Qi, and Matthias Nießner. Shape completion using 3D encoder-predictor CNNs and shape synthesis. In *CVPR*, pages 5868–5877, 2017. 2

[12] Philipp Erler, Paul Guerrero, Stefan Ohrhallinger, Niloy J Mitra, and Michael Wimmer. Points2Surf: Learning implicit surfaces from point clouds. In *ECCV*, pages 108–124, 2020. 1, 2

[13] Xifeng Gao, Kui Wu, and Zherong Pan. Low-poly mesh generation for building models. In *ACM SIGGRAPH 2022 Conference Proceedings*, New York, NY, USA, 2022. Association for Computing Machinery. 2

[14] Michael Garland and Paul S Heckbert. Surface simplification using quadric error metrics. In *Proceedings of the 24th annual conference on Computer graphics and interactive techniques*, pages 209–216, 1997. 2, 5, 7

[15] Jiahui Huang, Hao-Xiang Chen, and Shi-Min Hu. A neural Galerkin solver for accurate surface reconstruction. *ACM TOG*, 41(6):1–16, 2022. 2

[16] Jin Huang, Jantien Stoter, Ravi Peters, and Liangliang Nan. City3D: Large-scale building reconstruction from airborne LiDAR point clouds. *Remote Sensing*, 14(9), 2022. 2

[17] Jiahui Huang, Zan Gojcic, Matan Atzmon, Or Litany, Sanja Fidler, and Francis Williams. Neural kernel surface reconstruction. In *CVPR*, pages 4369–4379, 2023. 2

[18] Zhangjin Huang, Yuxin Wen, Zihao Wang, Jinjuan Ren, and Kui Jia. Surface reconstruction from point clouds: A survey and a benchmark. *IEEE TPAMI*, 2024. 1, 2

[19] Michael Kazhdan and Hugues Hoppe. Screened Poisson surface reconstruction. *ACM TOG*, 32(3):1–13, 2013. 1, 2, 5

[20] Sebastian Koch, Albert Matveev, Zhongshi Jiang, Francis Williams, Alexey Artemov, Evgeny Burnaev, Marc Alexa, Denis Zorin, and Daniele Panozzo. ABC: A big CAD model dataset for geometric deep learning. In *CVPR*, pages 9601–9611, 2019. 5, 1, 3

[21] Harold W Kuhn. The Hungarian method for the assignment problem. *Naval research logistics quarterly*, 2(1-2):83–97, 1955. 4

[22] Minglei Li and Liangliang Nan. Feature-preserving 3D mesh simplification for urban buildings. *ISPRS Journal of Photogrammetry and Remote Sensing*, 173:135–150, 2021. 2

[23] Pu Li, Jianwei Guo, Xiaopeng Zhang, and Dong-Ming Yan. SECAD-Net: Self-supervised CAD reconstruction by learning sketch-extrude operations. In *CVPR*, pages 16816–16826, 2023. 2, 5, 7, 1

[24] Shanshan Li, Pan Gao, Xiaoyang Tan, and Mingqiang Wei. ProxyFormer: Proxy alignment assisted point cloud completion with missing part sensitive transformer. In *CVPR*, pages 9466–9475, 2023. 2

[25] Yujia Liu, Anton Obukhov, Jan Dirk Wegner, and Konrad Schindler. Point2CAD: Reverse engineering CAD models from 3D point clouds. In *CVPR*, pages 3763–3772, 2024. 2

[26] Liangliang Nan and Peter Wonka. PolyFit: Polygonal surface reconstruction from point clouds. In *ICCV*, pages 2353–2361, 2017. 1, 2, 5, 6, 7

[27] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. PointNet: Deep learning on point sets for 3D classification and segmentation. In *CVPR*, pages 652–660, 2017. 2

[28] Charles R. Qi, Li Yi, Hao Su, and Leonidas J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. In *NeurIPS*, 2017. 2

[29] Daxuan Ren, Jianmin Zheng, Jianfei Cai, Jiatong Li, Haiyong Jiang, Zhongang Cai, Junzhe Zhang, Liang Pan, Mingyuan Zhang, Haiyu Zhao, et al. CSG-Stump: A learning friendly CSG-like representation for interpretable shape parsing. In *ICCV*, pages 12478–12487, 2021. 2

[30] D. Salinas, F. Lafarge, and P. Alliez. Structure-aware mesh decimation. *Comput. Graph. Forum*, 34(6):211–227, 2015. 2

[31] Ruwen Schnabel, Roland Wahl, and Reinhard Klein. Efficient RANSAC for point-cloud shape detection. *Comput. Graph. Forum*, 26(2):214–226, 2007. 2

[32] Raphael Sulzer and Florent Lafarge. Concise plane arrangements for low-poly surface and volume modelling. *ECCV*, 2024. 1, 2, 5, 6, 7

[33] Junshu Tang, Zhijun Gong, Ran Yi, Yuan Xie, and Lizhuang Ma. LAKe-Net: Topology-aware point cloud completion by localizing aligned keypoints. In *CVPR*, pages 1726–1735, 2022. 2

[34] Keneni W Tesema, Lyndon Hill, Mark W Jones, Muneeb I Ahmad, and Gary KL Tam. Point cloud completion: A survey. *IEEE TVCG*, 2023. 2

[35] Hugues Thomas, Charles R. Qi, Jean-Emmanuel Deschaud, Beatriz Marcotegui, François Goulette, and Leonidas J. Guibas. KPConv: Flexible and deformable convolution for point clouds. In *ICCV*, pages 6411–6420, 2019. 2

[36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, page 6000–6010, 2017. 3

[37] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E. Sarma, Michael M. Bronstein, and Justin M. Solomon. Dynamic graph CNN for learning on point clouds. *ACM TOG*, 38(5): 146:1–146:12, 2019. 2

[38] Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 3D ShapeNets: A deep representation for volumetric shapes. In *CVPR*, pages 1912–1920, 2015. 2

[39] Xuhao Xiang, Chaoping Zhang, Zizhuang Wei, and Shenghua Gao. SnowflakeNet: Point cloud completion by snowflake point deconvolution with skip-transformer. In *ICCV*, pages 5499–5509, 2021. 2

[40] Haozhe Xie, Hongxun Yao, Shangchen Zhou, Jiageng Mao, Shengping Zhang, and Wenxiu Sun. GRNet: Gridding residual network for dense point cloud completion. In *ECCV*, pages 365–381, 2020. 2, 5, 6, 8

[41] Xuejun Yan, Hongyu Yan, Jingjing Wang, Hang Du, Zhihong Wu, Di Xie, Shiliang Pu, and Li Lu. FBNet: Feedback network for point cloud completion. In *ECCV*, pages 676–693, 2022. 2

[42] Yaoqing Yang, Chen Feng, Yiru Shen, and Dong Tian. FoldingNet: Point cloud auto-encoder via deep grid deformation. In *CVPR*, pages 206–215, 2018. 2, 5, 6, 8

[43] Fenggen Yu, Zhiqin Chen, Manyi Li, Aditya Sanghi, Hooman Shayani, Ali Mahdavi-Amiri, and Hao Zhang. CAPRI-Net: Learning compact CAD shapes with adaptive primitive assembly. In *CVPR*, pages 11768–11778, 2022. 2

[44] Lequan Yu, Xianzhi Li, Chi-Wing Fu, Daniel Cohen-Or, and Pheng-Ann Heng. PU-Net: Point cloud upsampling network. In *CVPR*, pages 2790–2799, 2018. 4

[45] Mulin Yu and Florent Lafarge. Finding good configurations of planar primitives in unorganized point clouds. In *CVPR*, pages 6367–6376, 2022. 2, 5

[46] Xumin Yu, Yongming Rao, Ziyi Wang, Zuyan Liu, Jiwen Lu, and Jie Zhou. PoinTr: Diverse point cloud completion with geometry-aware transformers. In *ICCV*, pages 12498–12507, 2021. 1, 2, 5, 6, 8

[47] Xumin Yu, Yongming Rao, Ziyi Wang, Jiwen Lu, and Jie Zhou. AdaPoinTr: Diverse point cloud completion with adaptive geometry-aware transformers. In *IEEE TPAMI*, pages 14114–14130, 2023. 2, 3, 5, 6, 8

[48] Wentao Yuan, Tejas Khot, David Held, Christoph Mertz, and Martial Hebert. PCN: Point completion network. In *2018 international conference on 3D vision (3DV)*, pages 728–737, 2018. 1, 2, 5, 6, 8

[49] Ruibin Zhao, Xinhai Liu, Jiancheng Li, and Hongbo Fu. SeedFormer: Patch seeds based point cloud completion with upsample transformer. In *CVPR*, pages 11967–11976, 2022. 2

[50] Zhe Zhu, Liangliang Nan, Haoran Xie, Honghua Chen, Jun Wang, Mingqiang Wei, and Jing Qin. CSDN: Cross-modal shape-transfer dual-refinement network for point cloud completion. *IEEE TVCG*, 2023. 2